Nama : Frederico Steven Kwok

NPM : 242310037

Kelas : TI-24-PA1

Mata Kuliah : Lab. Pemrograman Web

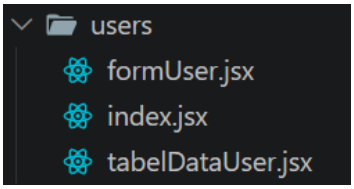
Tugas : Pertemuan 10 – Tugas dan Latihan 8

Github : https://github.com/RicoSteven120206/PW-MERN-STACK-242310037.git

Dokumentasi Lengkap Source Code Berada di Repository Github

1. Berdasarkan latihan pembelajaran diatas, integrasikan Rest API untuk management user pada frontend apss kalian dengan nama menu Users, berdasarkan api users yang telah dikerjakan.

   ⇨ Directory pada Struktur untuk membuat integrasikan Rest API untuk management user pada frontend



Pada Source Code users/formUser.jsx

```
"use client";

import React, { useEffect, useState } from "react";
import { Button } from "@/components/ui/button";
import {
  CREATE_USER,
  UPDATE_USER,
  GET_USER_BY_ID,
} from "@/components/apis/UserServices";
import { openModal, ModalResponse } from "@/components/ui/modals";

export default function FormUser({ user_id, ReloadUser }) {
```

```
  const [formData, setFormData] = useState({
    username: "",
    email: "",
    password: "",
    is_active: true,
  });
  const [loading, setLoading] = useState(false);

  // Ambil detail data jika dalam mode Edit
  useEffect(() => {
    if (user_id) {
      setLoading(true);
      GET_USER_BY_ID(user_id)
        .then((res) => {
          if (res && (res.success || res.data)) {
            const data = res.data || res;
            setFormData({
              username: data.username || "",
              email: data.email || "",
              password: "", // Kosongkan password saat edit
              is_active: data.is_active ?? true,
            });
          }
        })
        .catch((err) => console.error("Error fetching user detail:",
err))
        .finally(() => setLoading(false));
    }
  }, [user_id]);

  const handleSubmit = async (e) => {
```

```
    e.preventDefault();
    setLoading(true);

    try {
      let res;
      const payload = { ...formData };

     // Jika mode Edit dan password tidak diisi, hapus dari payload
agar tidak mengubah password lama
      if (user_id && !payload.password) {
        delete payload.password;
      }

      if (user_id) {
        res = await UPDATE_USER(user_id, payload);
      } else {
        res = await CREATE_USER(payload);
      }

        if (res && (res.success || res.statusCode === 200 ||
res.statusCode === 201)) {
        openModal({
          message: (
            <ModalResponse
                message={`User successfully ${user_id ? "updated" :
"created"}!`}
              title="Success"
            />
          ),
        });
        if (ReloadUser) ReloadUser();
```

```
      } else {
        openModal({
          message: (
            <ModalResponse
              message={res?.message || "Failed to save user"}
              title="Error"
              variant="danger"
            />
          ),
        });
      }
    } catch (err) {
      openModal({
        message: (
          <ModalResponse
            message={err?.message || "Something went wrong"}
            title="Error"
            variant="danger"
          />
        ),
      });
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="p-2">
      <h4 className="fw-bold mb-4">{user_id ? "Edit User" : "Add
New User"}</h4>
```

```
    {/* Field Username */}
    <div className="mb-3">
            <label className="form-label fw-semibold text-
secondary">Username</label>
      <input
        type="text"
        className="form-control"
        placeholder="Enter username"
        value={formData.username}
         onChange={(e) => setFormData({ ...formData, username:
e.target.value })}
        required
      />
    </div>

    {/* Field Email */}
    <div className="mb-3">
            <label className="form-label fw-semibold text-
secondary">Email Address</label>
      <input
        type="email"
        className="form-control"
        placeholder="Enter email address"
        value={formData.email}
            onChange={(e) => setFormData({ ...formData, email:
e.target.value })}
        required
      />
    </div>

    {/* Field Status Active */}
```

```
    <div className="mb-3">
            <label className="form-label fw-semibold text-
secondary">Status</label>
      <select
       className="form-select"
       value={formData.is_active ? "true" : "false"}
       onChange={(e) =>
          setFormData({ ...formData, is_active: e.target.value ===
"true" })
       }
      >
       <option value="true">Active</option>
       <option value="false">Inactive</option>
      </select>
    </div>

    {/* Field Password */}
    <div className="mb-4">
            <label className="form-label fw-semibold text-
secondary">
             Password {user_id && <small className="text-
muted">(Leave blank if unchanged)</small>}
      </label>
      <input
       type="password"
       className="form-control"
       placeholder="Enter password"
       value={formData.password}
        onChange={(e) => setFormData({ ...formData, password:
e.target.value })}
       required={!user_id}
```

```
        />
      </div>

      <div className="d-flex justify-content-end gap-2 border-top
pt-3">
                  <Button variant="primary" type="submit"
disabled={loading} className="px-4">
          {loading ? "Saving..." : user_id ? "Update User" : "Save
User"}
        </Button>
      </div>
    </form>
  );
}
```

Pada Source Code users/tabelDataUser.jsx

```
"use client";

import React, { useMemo, useState } from "react";
import { Cards } from "@/components/ui/cards";
import { Button } from "@/components/ui/button";
import {
  HeaderDatatables,
  SearchInput,
  PaginationComponent,
} from "@/components/ui/datatables";
import { openModal, ModalResponse } from
"@/components/ui/modals";
import { DELETE_USER } from
"@/components/apis/UserServices";
import FormUser from "./formUser";
```

```
export default function TabledataUser({ data = [], ReloadData })
{
  const [search, setSearch] = useState("");
  const [sorting, setSorting] = useState({ field: "", order: "" });
  const [currentPage, setCurrentPage] = useState(1);
  const ITEMS_PER_PAGE = 10;

  // Header diselaraskan dengan field dari Controller Sequelize
  const table_headers = [
    { name: "NO", field: "id", sortable: false, className: "ps-4 pe-
2 py-3 text-center align-middle bg-light text-secondary fw-
semibold border-bottom" },
     { name: "USERNAME", field: "username", sortable: true,
className: "px-3 py-3 align-middle bg-light text-secondary fw-
semibold border-bottom" },
    { name: "EMAIL", field: "email", sortable: true, className: "px-
3 py-3 align-middle bg-light text-secondary fw-semibold border-
bottom" },
    { name: "STATUS", field: "is_active", sortable: true, className:
"px-3 py-3 text-center align-middle bg-light text-secondary fw-
semibold border-bottom" },
    { name: "ACTIONS", field: "id", sortable: false, className: "ps-
2 pe-4 py-3 text-center align-middle bg-light text-secondary fw-
semibold border-bottom" },
  ];

  const handleAdd = () => {
    openModal({
      message: <FormUser ReloadUser={ReloadData} />,
      size: "lg",
    });
```

```
  };

  const handleEdit = (user) => {
    openModal({
                        message:        <FormUser        user_id={user.id}
ReloadUser={ReloadData} />,
      size: "lg",
    });
  };

  const handleDelete = async (userId) => {
     if (window.confirm("Are you sure you want to delete this
user?")) {
      try {
        const res = await DELETE_USER(userId);
        if (res && (res.success || res.statusCode === 200)) {
          openModal({
                message: <ModalResponse message="User deleted
successfully!" title="Success" />,
          });
          if (ReloadData) ReloadData();
        } else {
          openModal({
              message: <ModalResponse message={res?.message ||
"Failed to delete user"} title="Error" variant="danger" />,
          });
        }
      } catch (err) {
        openModal({
            message: <ModalResponse message={err?.message ||
"Error deleting user"} title="Error" variant="danger" />,
```

```
      });
    }
  }
};

const ResultData = useMemo(() => {
  let computedData = Array.isArray(data) ? [...data] : [];

  if (search) {
    computedData = computedData.filter((listData) => {
      return Object.keys(listData).some((key) => {
        const value = listData[key];
                                return value != null &&
String(value).toLowerCase().includes(search.toLowerCase());
      });
    });
  }

  if (sorting.field) {
    const reversed = sorting.order === "asc" ? 1 : -1;
    computedData.sort((a, b) => {
      const valA = a[sorting.field] || "";
      const valB = b[sorting.field] || "";
      return reversed * String(valA).localeCompare(String(valB));
    });
  }

  return computedData;
}, [data, search, sorting]);

const totalitems = ResultData.length;
```

```
 const paginatedData = useMemo(() => {
  const start = (currentPage - 1) * ITEMS_PER_PAGE;
  return ResultData.slice(start, start + ITEMS_PER_PAGE);
 }, [ResultData, currentPage]);

 return (
    <Cards className="shadow-sm border rounded-3 mt-4 overflow-hidden bg-white">
      <Cards.Header className="px-4 py-3 bg-white border-bottom flex-wrap gap-3">
    <div className="d-flex align-items-center gap-3">
        <h5 className="fw-bold fs-4 mb-0 text-dark">User Management</h5>
        <Button variant="primary" className="btn-sm px-3 rounded-2" onClick={handleAdd}>
       <i className="bi bi-plus-lg me-1"></i> Add User
      </Button>
    </div>
    <div style={{ width: "280px" }}>
      <SearchInput
        keyword={search}
        onAction={(e) => {
          setSearch(e.target.value);
          setCurrentPage(1);
        }}
      />
    </div>
   </Cards.Header>

   <Cards.Body className="p-0">
```

```
        <div className="table-responsive">
          <table className="table table-hover align-middle mb-0">
            <HeaderDatatables
              headers={table_headers}
              onSorting={(field, order) => setSorting({ field, order })}
            />
            <tbody>
              {paginatedData.length > 0 ? (
                paginatedData.map((user, index) => (
                  <tr key={user.id || index}>
                      <td className="ps-4 pe-2 py-3 text-center align-middle text-muted fs-6">
                        {(currentPage - 1) * ITEMS_PER_PAGE + index + 1}
                      </td>
                      <td className="px-3 py-3 align-middle">
                                  <strong className="text-dark fs-6">{user.username}</strong>
                      </td>
                     <td className="px-3 py-3 align-middle text-secondary fs-6">
                        {user.email}
                      </td>
                      <td className="px-3 py-3 text-center align-middle">
                          <span className={`badge px-3 py-2 fw-normal rounded-2 ${user.is_active ? 'bg-success-subtle text-success border border-success-subtle' : 'bg-danger-subtle text-danger border border-danger-subtle'}`}>
                            {user.is_active ? "Active" : "Inactive"}
                          </span>
                      </td>
                        <td className="ps-2 pe-4 py-3 text-center align-
```

```
middle">
                  <div className="d-flex justify-content-center gap-
2">
                  <Button
                    variant="warning"
                    outline
                     className="btn-sm p-2 d-inline-flex align-items-
center justify-content-center rounded-2"
                    onClick={() => handleEdit(user)}
                    title="Edit"
                  >
                    <i className="bi bi-pencil"></i>
                  </Button>
                  <Button
                    variant="danger"
                    outline
                     className="btn-sm p-2 d-inline-flex align-items-
center justify-content-center rounded-2"
                    onClick={() => handleDelete(user.id)}
                    title="Delete"
                  >
                    <i className="bi bi-trash"></i>
                  </Button>
                </div>
              </td>
            </tr>
          ))
        ) : (
          <tr>
            <td colSpan="5" className="text-center py-5">
              <i className="bi bi-people fs-1 text-muted d-block
```

```
mb-3"></i>
                    <p  className="text-muted  mb-0">No  users
found</p>
                </td>
              </tr>
            )}
          </tbody>
        </table>

        {totalitems > 0 && (
          <div className="d-flex align-items-center justify-content-
center py-3 px-4 border-top bg-white">
            <PaginationComponent
              total={totalitems}
              itemsPerPage={ITEMS_PER_PAGE}
              currentPage={currentPage}
              onPageChange={(page) => setCurrentPage(page)}
            />
          </div>
        )}
      </div>
    </Cards.Body>
  </Cards>
 );
}
```

Pada Source Code users/index.jsx

```
"use client";

import React, { useEffect, useState, useCallback } from "react";
import TabledataUser from "./tabelDataUser";
import          {          GET_ALL_USER          }          from
```

```
"@/components/apis/UserServices";

export default function UsersPage() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  const fetchUsers = useCallback(async () => {
    setLoading(true);
    try {
      const res = await GET_ALL_USER();
          if (res && (res.success || Array.isArray(res.data) ||
Array.isArray(res))) {
        setUsers(res.data || res);
      }
    } catch (error) {
      console.error("Failed to fetch users:", error);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    fetchUsers();
  }, [fetchUsers]);

  return (
    <div className="container-fluid py-4">
      {loading ? (
        <div className="text-center py-5">
                  <div  className="spinner-border  text-primary"
role="status">
```

```
            <span className="visually-hidden">Loading...</span>
          </div>
        </div>
      ) : (
        <TabledataUser data={users} ReloadData={fetchUsers} />
      )}
    </div>
  );
}
```

2. Buatlah validasi keamanan dengan kondisi jika user telah melakukan sign-in kedalam aplikasi, maka url path /sign-in atau /sign-up tidak bisa kembali diakses melainkan di redirect ke /cms.

   ⇨ Terdapat pada Middleware untuk validasi mendapatkan jsonwebtoken, jika jsonwebtoken tersebut expired atau dihapus, maka hak akses tersebut sudah berakhir, dan ada terdapat pada Source Code di frontend untuk mengatur hak akses routes. Berikut Source Code lengkap.

   Source Code backend/middleware/auth.js

```
const jwt = require('jsonwebtoken');

exports.verifyToken = (req, res, next) => {
  try {
    // Get token from header
    const authHeader = req.headers["authorization"];
    const token = authHeader && authHeader.split(" ")[1];

    if (!token) {
      return res.status(401).json({
        success: false,
        message: "Access token is required",
      });
    }
```

```
    // Verify token
    jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
      if (err) {
        if (err.name === "TokenExpiredError") {
          return res.status(401).json({
            success: false,
            message: "Token has expired",
          });
        }
        return res.status(403).json({
          success: false,
          message: "Invalid token",
        });
      }

      // Save user info to request
      req.user = decoded;
      next();
    });
  } catch (error) {
    console.error("Error verifying token:", error);
    res.status(500).json({
      success: false,
      message: "Failed to authenticate token",
      error: error.message,
    });
  }
};

exports.checkUserActive = async (req, res, next) => {
```

```
    try {
      const db = require("../models");
      const User = db.User;

      const user = await User.findByPk(req.user.id);

      if (!user) {
        return res.status(404).json({
          success: false,
          message: "User not found",
        });
      }

      if (!user.is_active) {
        return res.status(403).json({
          success: false,
          message: "Account is inactive",
        });
      }

      next();
    } catch (error) {
      console.error("Error checking user status:", error);
      res.status(500).json({
        success: false,
        message: "Failed to verify user status",
        error: error.message,
      });
    }
  };
```

Source Code frontend/app/sign-in.jsx

```
"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";
import { useAuth } from "@/context/AuthContext";
import "@/components/auth/auth.css";
import { TextInput, TextInputPassword } from "@/components/ui/forms";
import { withAuthRedirect } from "@/components/auth/withAuthRedirect";
import { LOGIN_USER } from "@/components/apis/UserServices";

function SignInPage() {
  const router = useRouter();
  const { login } = useAuth();
  const [formData, setFormData] = useState({
    email: "",
    password: "",
  });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
    setError("");
  };
```

```
  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");

    if (!formData.email || !formData.password) {
      setError("Email and password are required");
      return;
    }

    setLoading(true);

    try {
      const result = await LOGIN_USER({
        email: formData.email,
        password: formData.password,
      });

      if (result.success) {
          // Simpan data user, accessToken, dan expiresIn ke AuthContext & LocalStorage
        await login(result.data, result.accessToken, result.expiresIn);

                                    const redirectUrl = sessionStorage.getItem("redirectAfterLogin");
        if (redirectUrl) {
          sessionStorage.removeItem("redirectAfterLogin");
          router.push(redirectUrl);
        } else {
          router.push("/cms/books");
        }
      } else {
```

```
        setError(result.message || "Login failed");
      }
    } catch (err) {
      console.error("Error during login:", err);
      setError("An error occurred. Please try again.");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="signin-container">
      <div className="signin-card">
        <h1 className="signin-title">Welcome Back</h1>
        <p className="signin-subtitle">Sign in to your account</p>

        {error && <div className="error-message">{error}</div>}

        <form onSubmit={handleSubmit} className="signin-form">
          <TextInput
            title="Email"
            required={true}
            id="email"
            name="email"
            value={formData.email}
            onChange={handleChange}
            placeholder="Enter your email"
          />

          <TextInputPassword
            required={true}
```

```
            title="Password"
            id="password"
            name="password"
            value={formData.password}
            onChange={handleChange}
            placeholder="Enter your password"
          />

                <button   type="submit"   className="signin-button"
disabled={loading}>
            {loading ? "Signing in..." : "Sign In"}
          </button>
        </form>

        <div className="signin-footer">
          <p>
            Do not have an account?{" "}
            <Link href="/sign-up" className="signup-link">
              Sign up here
            </Link>
          </p>
        </div>
      </div>
    </div>
  );
}

export default withAuthRedirect(SignInPage);
```

Source Code frontend/app/sign-up.jsx

```
"use client";
```

```
import { useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";
import "@/components/auth/auth.css";
import { TextInput, TextInputPassword } from "@/components/ui/forms";
import { REGISTER_USER } from "@/components/apis/UserServices"; // Sesuaikan API service kamu

export default function SignUpPage() {
  const router = useRouter();
  const [formData, setFormData] = useState({
    username: "",
    email: "",
    password: "",
  });
  const [error, setError] = useState("");
  const [successMsg, setSuccessMsg] = useState("");
  const [loading, setLoading] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
    setError("");
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setSuccessMsg("");
```

```
        if (!formData.username || !formData.email || !formData.password) {
      setError("Semua bidang wajib diisi");
      return;
    }

    setLoading(true);

    try {
      const result = await REGISTER_USER(formData);

      if (result.success) {
          setSuccessMsg("Pendaftaran berhasil! Mengalihkan ke halaman login...");

        // JANGAN LANGSUNG LOGIN! Berikan delay sebentar lalu redirect ke /sign-in
        setTimeout(() => {
          router.push("/sign-in");
        }, 1500);
      } else {
        setError(result.message || "Pendaftaran gagal");
      }
    } catch (err) {
      console.error("Error during register:", err);
      setError("Terjadi kesalahan. Silakan coba lagi.");
    } finally {
      setLoading(false);
    }
  };
```

```
 return (
  <div className="signup-container">
   <div className="signup-card">
    <h1 className="signup-title">Create Account</h1>
    <p className="signup-subtitle">Sign up to get started</p>

    {error && <div className="error-message">{error}</div>}
    {successMsg && (
                <div  className="error-message"  style={{
backgroundColor: "#d1e7dd", borderColor: "#badbcc", color:
"#0f5132" }}>
      {successMsg}
     </div>
    )}

      <form  onSubmit={handleSubmit}  className="signup-
form">
     <TextInput
      title="Username"
      required={true}
      id="username"
      name="username"
      value={formData.username}
      onChange={handleChange}
      placeholder="Enter your username"
     />

     <TextInput
      title="Email"
      required={true}
```

```
            id="email"
            name="email"
            value={formData.email}
            onChange={handleChange}
            placeholder="Enter your email"
          />

          <TextInputPassword
            required={true}
            title="Password"
            id="password"
            name="password"
            value={formData.password}
            onChange={handleChange}
            placeholder="Enter your password"
          />

              <button type="submit" className="signup-button"
disabled={loading}>
            {loading ? "Registering..." : "Sign Up"}
          </button>
        </form>

        <div className="signup-footer">
          <p>
            Already have an account?{" "}
            <Link href="/sign-in" className="login-link">
              Sign in here
            </Link>
          </p>
        </div>
```

```
      </div>
    </div>
  );
}
```

Source Code frontend/components/auth/ProtectedRoutes.jsx

```
"use client";

import { useEffect, useState } from "react";
import { useRouter, usePathname } from "next/navigation";
import { useAuth } from "@/context/AuthContext";
import { Skeleton } from "@/components/ui/loading";

export default function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  const router = useRouter();
  const pathname = usePathname();
  const [shouldRedirect, setShouldRedirect] = useState(false);

  useEffect(() => {
    if (!loading && !user) {
      // Tambahkan delay kecil sebelum redirect
      const timer = setTimeout(() => {
        setShouldRedirect(true);
      }, 100);

      return () => clearTimeout(timer);
    }
  }, [user, loading]);

  useEffect(() => {
    if (shouldRedirect) {
```

```
            sessionStorage.setItem("redirectAfterLogin", pathname);
            router.push("/sign-in");
        }
    }, [shouldRedirect, router, pathname]);

    if (loading) {
        return (
            <Skeleton />
        );
    }

    if (!user) {
        return null;
    }

    return <>{children}</>;
}
```

Source Code frontend/components/auth/withAuthRedirect.jsx

```
"use client";

import { createContext, useContext, useState, useEffect } from
"react";
import { useRouter } from "next/navigation";

const AuthContext = createContext({});

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const router = useRouter();
```

```
useEffect(() => {
  checkAuth();
}, []);

const checkAuth = () => {
  try {
    const token = localStorage.getItem("accessToken");
    const userData = localStorage.getItem("user");
    const expiresIn = localStorage.getItem("expiresIn");

    if (token && userData && expiresIn) {
      const currentTime = Math.floor(Date.now() / 1000);
      const expirationTime = parseInt(expiresIn, 10);

      if (currentTime >= expirationTime) {
        console.log("Token expired, logging out...");
        logout();
        return;
      }
      setUser(JSON.parse(userData));
    } else {
      setUser(null);
    }
  } catch (error) {
    console.error("Error checking auth:", error);
    setUser(null);
  } finally {
    setLoading(false);
  }
};
```

```
  const login = (userData, token, expiresIn) => {
    localStorage.setItem("accessToken", token);
    localStorage.setItem("expiresIn", expiresIn);
    localStorage.setItem("user", JSON.stringify(userData));
    setUser(userData);

    return Promise.resolve();
  };

  const logout = () => {
    setUser(null);

    // Clears all storage & cookies
    localStorage.removeItem("accessToken");
    localStorage.removeItem("expiresIn");
    localStorage.removeItem("user");
    localStorage.clear();
    sessionStorage.clear();

     document.cookie = "accessToken=; path=/; expires=Thu, 01
Jan 1970 00:00:01 GMT;";
    document.cookie = "token=; path=/; expires=Thu, 01 Jan 1970
00:00:01 GMT;";

    window.location.href = "/sign-in";
  };

  return (
     <AuthContext.Provider value={{ user, loading, login, logout,
checkAuth }}>
      {children}
```

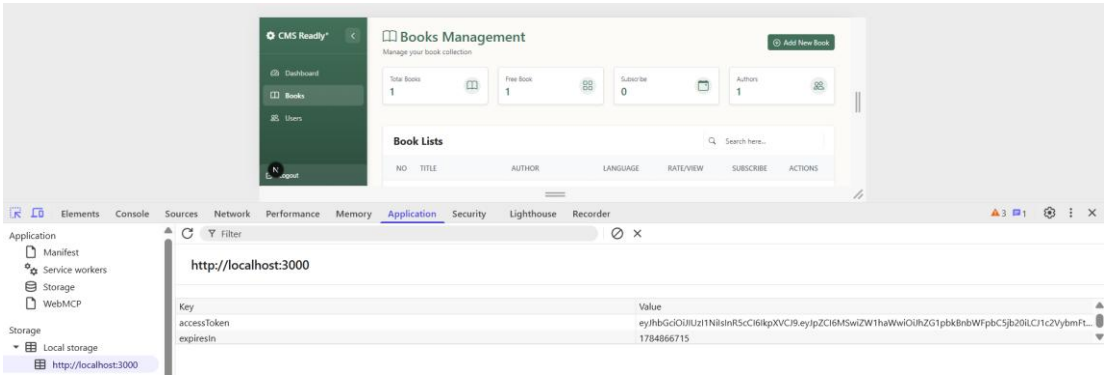
```
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
        throw new Error("useAuth must be used within
AuthProvider");
  }
  return context;
};
```

Source Code frontend/context/AuthContext.jsx

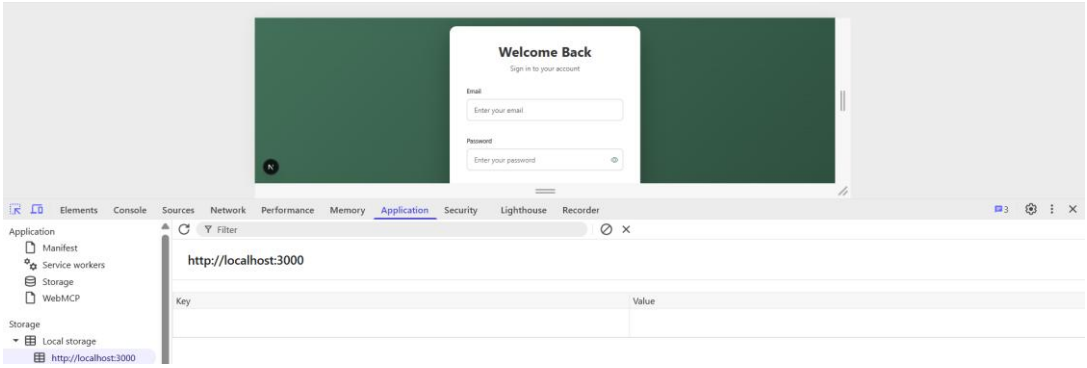
```
"use client";

import { useEffect } from "react";
import { useRouter } from "next/navigation";
import { useAuth } from "@/context/AuthContext";

export const withAuthRedirect = (Component) => {
  return function WithAuthRedirectComponent(props) {
    const { user, loading } = useAuth();
    const router = useRouter();

    useEffect(() => {
      if (!loading && user) {
        router.push("/cms/books");
      }
    }, [user, loading, router]);

    if (loading) {
```

```
      return (
        <div
              className="d-flex  justify-content-center  align-items-
center"
          style={{ minHeight: "100vh" }}
        >
          <div classNam="spinner-border" role="status">
            <span classNam="visually-hidden">Loading...</span>
          </div>
        </div>
      );
    }

    if (user) {
      return null;
    }

    return <Component {...props} />;
  };
};
```

Tersimpan didalam Local Storage pada token dari JWT, jika ada berhasil masuk ke dalam Website Hak akses tertentu.



Apabila JWT nya dihapus, maka akan terjadi perpindahan ke Sign-In untuk melakukan login/verifikasi akun users Kembali.



Bagian yang terkait mengelola tersebut berada dibawah berikut.

Source Code lib/apiClient.jsx

```
import axios from "axios";

const ApiClient = axios.create({
    baseURL:    process.env.NEXT_PUBLIC_BACKEND_URI    ||
"http://localhost:3001",
 headers: {
   "Content-Type": "application/json",
 },
});

ApiClient.interceptors.request.use((config) => {
 if (typeof window !== "undefined") {
   const token = localStorage.getItem("accessToken");
   if (token) {
     config.headers.Authorization = `Bearer ${token}`;
   }
 }
 return config;
});

ApiClient.interceptors.response.use(
```

```
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      if (typeof window !== "undefined") {
        localStorage.removeItem("accessToken");
        window.location.href = "/sign-in";
      }
    }
    return Promise.reject(error);
  }
);

export default ApiClient;
```

Source Code components/apis/UserService.jsx

```
import ApiClient from "@/lib/apiClient";

export const LOGIN_USER = async (credentials) => {
   const response = await ApiClient.post("/api/users/login",
credentials);
  return response.data;
};

export const REGISTER_USER = async (userData) => {
   const response = await ApiClient.post("/api/users/register",
userData);
  return response.data;
};

export const GET_ALL_USER = async () => {
  const response = await ApiClient.get("/api/users");
  return response.data;
```

```
};

export const GET_USER_BY_ID = async (user_id) => {
  const response = await ApiClient.get(`/api/users/${user_id}`);
  return response.data;
};

export const CREATE_USER = async (payload) => {
  const response = await ApiClient.post("/api/users", payload);
  return response.data;
};

export const UPDATE_USER = async (user_id, payload) => {
   const response = await ApiClient.put(`/api/users/${user_id}`,
payload);
  return response.data;
};

export const PATCH_USER = async (user_id, payload) => {
  const response = await ApiClient.patch(`/api/users/${user_id}`,
payload);
  return response.data;
};

export const DELETE_USER = async (user_id) => {
            const response = await
ApiClient.delete(`/api/users/${user_id}`);
  return response.data;
};
```

Source Code app/cms/layout.jsx

```
'use client';

import React, { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import Sidebar from '../../components/cms/components/sidebar';
import Modals from '../../components/ui/modals';
import './cms.css';

export default function CMSLayout({ children }) {
  const router = useRouter();
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('accessToken');

    if (!token) {
      router.replace('/sign-in');
    } else {
      setIsAuthenticated(true);
      setLoading(false);
    }
  }, [router]);

  if (loading || !isAuthenticated) {
    return (
      <div className="d-flex justify-content-center align-items-center vh-100 bg-light">
        <div className="text-center">
```

```
          <div
            className="spinner-border text-primary mb-3"
            role="status"
            style={{ width: "3rem", height: "3rem" }}
          >
                                        <span       className="visually-
hidden">Authenticating...</span>
          </div>
           <p className="text-muted fw-semibold mb-0">Verifikasi
Hak Akses...</p>
        </div>
      </div>
    );
  }

  return (
    <>
      <div className="cms-container">
        <Sidebar />
        <main className="main-content p-4">{children}</main>
      </div>
      <Modals />
    </>
  );
}
```

3. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.

   ⇨ Done. Pengerjaan Dokumentasi Selesai.

4. Upload project tersebut pada GITHUB dengan menggunakan repository bernama "**PW-MERN-STACK-NPM**" dengan nama commit "**Praktikum 8**". Link GITHUB PW-MERN-STACK-242310037
   https://github.com/RicoSteven120206/PW-MERN-STACK-242310037.git